

백엔드 아키텍처 재검증 · Cleanup 품질 개선 · 프로덕션 배포 검증

Glocalizer · 작성일 2026-07-26

1. 배경 — 프로젝트 아키텍처 전면 교체 발견

사용자가 전달한 "프로젝트 종합 현황 보고서" PDF와 GitHub 저장소를 대조 확인하는 과정에서, 이전 세션까지 작업하던 NVIDIA(Nemotron OCR + GLM 5.2) 기반 백엔드가 팀원에 의해 **PaddleOCR + Groq Llama 3.3 70B** 기반으로 완전히 재작성되어 있는 것을 확인했다. Migration 009~012, Render/Docker 배포 설정, job worker heartbeat/lease, OCR 벤치마크 도구까지 갖춘 상태였다. 이 발견을 바탕으로 로컬 개발 환경을 새 아키텍처에 맞춰 복구하고, 사용자가 브라우저에서 실제 테스트하며 리포트한 버그를 근본 원인부터 수정했다.

2. 로컬 개발 환경 복구

- `.env` 를 새 스키마로 재작성 (팀원에게 받은 `GROQ_API_KEY`, Supabase 자격증명, `DATABASE_URL`)
- **DB 마이그레이션 정합성 문제 발견 및 해결**: 실제 DB 스키마에는 009~012의 효과(컬럼·인덱스)가 이미 반영돼 있었지만, 마이그레이션 추적 테이블(`app.schema_migrations`) 자체가 없어서 `npm run db:migrate` 가 반복 실패하던 것을 발견 — 체크섬을 계산해 12개 마이그레이션 전부를 추적 테이블에 baseline으로 기록해 동기화
- **로컬 PaddleOCR 환경 신규 구축**: 로컬 Python이 3.14인데 PaddlePaddle이 3.9~3.13까지만 지원해 설치 자체가 불가능했던 걸 발견 → Homebrew로 Python 3.12 설치 후 전용 venv 생성, PaddleOCR·OpenCV 등 설치 완료
- 세션 내내 `npm run dev` 를 반복 실행하며 정리 안 된 좀비 프로세스 4개(그중 하나는 지난 수요일부터 떠있음) 발견 후 전체 정리

3. 사용자 리포트 버그 수정

3.1 로딩 중 에디터 전체 UI가 먼저 보이는 문제

로딩 스피너가 캔버스 영역에만 오버레이로 떠서 상단 바·언어 탭·파일 목록·설정 패널이 로딩 중에도 그대로 노출/조작 가능했다. 처리 중(`isLoading`)에는 에디터 화면 자체를 렌더링하지 않고 전체화면 로딩 화면(스

피너 + 실시간 진행률 바 + 처리 단계 메시지 + 취소 버튼)만 보여주도록 수정했다.

3.2 배경 자동 정리 실패 — 근본 원인 규명

발견 처음엔 "cleanup 알고리즘이 약하다"고 가정했지만, 실측 결과 완전히 다른 곳에 진짜 버그가 있었다. "대박"처럼 2~3글자 짧은 한글 문구(이모티콘 문구 대부분이 여기 해당)는 항상 Vision 모델 (Groq/Gemini)에게 재판독을 시키는데, 이때 Vision이 텍스트는 정확히 읽으면서도 **자체적으로 추정**한 **bbox 좌표가 실제 글자와 전혀 겹치지 않는 빈 공간**을 가리켰다. 이 부정확한 위치를 그대로 믿고 지우려니 mask coverage가 0%로 나와 안전장치가 발동, 매번 manual-required로 떨어졌다.

조치 Vision 모델은 원래 목적대로 "글자 오독 교정"에만 쓰고, 위치(bbox)는 검출 전문 모델인 PaddleOCR이 찾은 걸 그대로 쓰도록 수정 (ocr-pipeline.service.ts).

결과 동일 이미지 기준 자동 cleanup 성공률 1/3 → 4/4로 개선. Supabase에 남아있던 **실제 사용자 업로드 이미지**로 재검증한 결과, 2건이 manual-required → 자동 성공으로 전환되는 것을 실제 이미지를 직접 열어 시각적으로 확인했다 (회색 말풍선·흰 배경 모두 이음매 없이 깔끔하게 제거됨).

4. Cleanup 알고리즘 업그레이드 — cv2.inpaint 도입과 한계

PaddleOCR 설치의 부수 의존성으로 OpenCV(cv2.inpaint)가 이미 들어와 있는 것을 발견하고, 기존의 조잡한 자체 선형보간 방식을 실제 사진편집 도구에서 쓰는 Telea 알고리즘으로 교체했다. Python bridge(inpaint_bridge.py)를 새로 만들어 연결했다.

지운 영역 방식	단순 배경(단색)	복잡 배경(그라데이션+반복무늬)
정밀 글자 마스크(색상 거리 기반)	정상 동작	배경이 여러 색이면 판별 자체가 무너져 글자가 절반 넘게 그대로 남음 (더 나쁜 실패)
OCR bbox 전체 사각형	정상 동작	글자는 확실히 지워지지만 반복무늬 위에 얼룩처럼 뭉개짐

한계 · 결정 그라데이션+물방울무늬 합성 테스트 이미지로 직접 두 방식을 비교 검증했다. "글자가 그대로 남는 것"이 "살짝 뭉개지는 것"보다 훨씬 나쁜 실패라고 판단해 **bbox 전체 사각형 방식을 기본값으로 확정**했다. 반복 패턴이 강한 배경을 완전히 자연스럽게 복원하려면 생성형 인페인팅(비용·지연 발생)이 필요한 별도 작업이며, 알고리즘 교체만으로는 못 넘는 한계선이라는 걸 명확히 확인했다.

5. 검증

- 백엔드 타입체크·빌드·테스트(120개) 전부 통과, 회귀 없음
- 실제 사용자 업로드 이미지 4장 재검증: 2장 자동 성공 전환 확인, 나머지 1장(2개 포맷)은 cleanup이 아닌 **OCR 신뢰도 게이트**(agreement_score 0.746, 자동승인 기준 0.82 미달)가 원인 — 별개의 정상 동작으로 확인

6. Git · 배포

- 수정사항을 `fix/ocr-bbox-cleanup-loading` 브랜치로 커밋·푸시 → PR #17 생성 → 팀원이 `main`에 merge
- **Render 프로덕션 배포 검증**: `https://glocalizer-api.onrender.com`에서 `health/readiness` 정상, 실제 이미지 3종(단순·실사용자·복잡배경)으로 종단 파이프라인 테스트 전부 통과. `cv2.inpaint`가 Docker 컨테이너 안에서도 정상 동작 확인. 연속 3회 AI 파이프라인 처리 후에도 서비스 안정적(재시작·크래시 없음), 보안 헤더·rate limit 정상
- **Vercel 프론트 ↔ Render 백엔드 연동 검증**: `https://glocalizer.vercel.app` 배포 확인, 번들 JS에 올바른 API 주소 확인, `FRONTEND_ORIGIN` 갱신 후 CORS를 프로젝트 생성부터 결과 조회까지 전 단계 실제 Origin 헤더로 재검증 — 전부 정상
- `UnderDeploy` 브랜치가 `main` 보다 7커밋 뒤쳐져 있던 것을 fast-forward로 동기화

7. 최종 판단

✓ **완료** 로딩 화면 버그, OCR bbox 오검출로 인한 cleanup 실패, 프론트-백엔드 실 배포 연동까지 전부 실제 이미지·실제 배포 환경 기준으로 검증 완료했다.

남은 과제: 반복 패턴이 강한 복잡 배경의 완전 자연스러운 복원은 생성형 인페인팅이 필요한 별도 작업이며, 정답 라벨이 있는 30장 규모의 OCR/cleanup 벤치마크로 실제 사용자 이미지 분포 기준 품질을 정량 측정하는 것이 다음 우선순위다.